



中国研究生创芯大赛  
CPIIC CHINA POSTGRADUATE IC  
INNOVATION COMPETITION

华为赛题一 · VVC 反变换模块（ITS）设计

# ITS VVC 反变换模块 详细设计文档



面向 H.266/VVC 的 500MHz 高频反变换引擎  
RTL 设计与实现

队伍名称：哈基米

## 目录

|                                             |    |
|---------------------------------------------|----|
| 1. 概述.....                                  | 1  |
| 1.1 项目背景.....                               | 1  |
| 1.2 设计目标.....                               | 1  |
| 1.3 标准参考.....                               | 2  |
| 2. 系统架构设计 .....                             | 2  |
| 2.1 顶层架构框图 .....                            | 2  |
| 3. 状态机设计 .....                              | 5  |
| 3.1 顶层状态机.....                              | 5  |
| 3.2 变换引擎内部状态机.....                          | 7  |
| 3.3 LFNST 模块状态机.....                        | 7  |
| 4. 模块详细设计 .....                             | 8  |
| 4.1 顶层模块 (its_top.v / its_core_500.v) ..... | 8  |
| 4.2 变换引擎 (its_transform_engine.v) .....     | 10 |
| 4.3 MAC 单元 (its_mac.v) .....                | 11 |
| 4.4 LFNST 模块 (its_lfnst.v) .....            | 11 |
| 4.5 异步 FIFO CDC 模块 .....                    | 13 |
| 5. 数据流与时序设计 .....                           | 14 |
| 5.1 完整数据流.....                              | 14 |
| 5.2 接口时序图.....                              | 14 |
| 6. ROM 结构设计 .....                           | 16 |

|                                       |    |
|---------------------------------------|----|
| 6.1 变换核 ROM (its_rom.v).....          | 16 |
| 6.2 LFNST ROM (its_lfnst_rom.v) ..... | 17 |
| 7. 定点量化方案 .....                       | 18 |
| 7.1 量化公式与位宽 .....                     | 18 |
| 7.2 溢出分析.....                         | 19 |
| 7.3 量化误差.....                         | 19 |
| 8. 存储资源汇总 .....                       | 20 |
| 9. 目标频率分析与实现结果 .....                  | 21 |
| 9.1 UltraScale+ 500MHz 实现结果.....      | 21 |
| 9.2 FPGA 评估口径说明 .....                 | 22 |
| 10. RTL 源代码清单 .....                   | 22 |

# 1. 概述

## 1.1 项目背景

本项目为第九届中国研究生创芯大赛华为赛题一：VVC (H.266) 视频编码标准的反变换模块 (Inverse Transform Subsystem, ITS)。VVC 作为新一代视频编码标准，相比 H.265/HEVC 提供了更高的压缩效率，其核心之一在于引入了更多种类的变换核和 LFNST 技术。ITS 模块负责在解码端将频域系数还原为时域残差信号，是视频解码器的关键组成部分。

本文档涵盖 ITS 模块的接口协议、总体架构、核心计算单元、LFNST 支持、定点量化方案、验证结果以及 FPGA 实现结果。最终赛题提交入口为 02\_rtl/rtl/its\_top.v，内部例化 its\_top\_500\_singleclk，端口与赛题 its\_top 单时钟接口保持一致；its\_top\_500\_wrapper 作为双时钟 CDC 集成结构保留，用于系统级集成参考。

本设计主要面向三类约束进行架构划分：一是 VVC 反变换需要同时支持 DCT2、DCT8、DST7 以及多种 TU 尺寸；二是 LFNST 对低频 4x4 区域具有独立的写回映射要求；三是 500MHz 目标周期仅 2ns，存储访问、ROM 地址生成、MAC 输入和输出反压路径均需要采用显式流水化和局部缓存设计。

## 1.2 设计目标

本设计在统一赛题接口下覆盖 DCT2、DCT8、DST7、LFNST、输入反压、输出反压和连续 TU 场景。输入侧按 it\_data\_addr 写入非零系数，输出侧每拍返回 4 个 10-bit 结果，并通过 it\_data\_out\_req 支持下游暂停接收。

本设计同时以功能正确性和 500MHz 可实现性作为设计约束。功能验证采用 its\_top\_500\_singleclk 的 1539 个用例和 its\_core\_500 的 94 个用例；时序评估采用 Kintex UltraScale+ xcku5p-ffvb676-2-e 器件上的实现报告作为 500MHz 达标依据。

| 目标项      | 具体要求                                                                          |
|----------|-------------------------------------------------------------------------------|
| 变换类型支持   | DCT2、DCT8、DST7 三种反变换核                                                         |
| 块大小覆盖    | DCT2: 4x4 ~ 64x64 (25 种) ; DCT8/DST7: 4x4 ~ 32x32 (各 16 种)                    |
| LFNST 支持 | nTrs=16 (任一边小于 8 的 TU) 和 nTrs=48 (宽高均不小于 8 的 TU) , 4 setIdx × 2 idx           |
| 计算性能     | 共享 1D transform engine 内含 4 个并行 MAC, 计算阶段并行处理 4 个输出点; 输出接口每拍打包 4 个 10-bit 结果。 |
| 接口协议     | 22-bit it_info, 符合赛题规范, 支持输入/输出反压                                             |
| 目标频率     | 500MHz                                                                        |
| 实现语言     | Verilog-2001 / SystemVerilog                                                  |

## 1.3 标准参考

变换核系数和 LFNST 系数在 RTL、测试向量和 Python reference model 中保持一致。ROM 系数由脚本统一生成并同步到 02\_rtl/rtl/ 与 03\_verification/tb/ 目录, 仿真与综合使用同一套数值来源, 避免 reference/golden 与 DUT 之间形成循环依赖。

针对赛题中关于二维处理顺序和连续 TU 输入能力的要求, 设计已实现先垂直后水平的二维反变换流程; 当上一 TU 输出尚未完成时, 输入侧仍可在 it\_data\_in\_req 允许后接收下一 TU。状态机通过 TU metadata queue 和 input closing 窗口机制保证连续 TU 的边界隔离。

- VVC 标准: ITU-T H.266 / ISO/IEC 23090-3 (JVET-S2001)
- LFNST 矩阵: 赛题附件 "Low frequency non.docx"

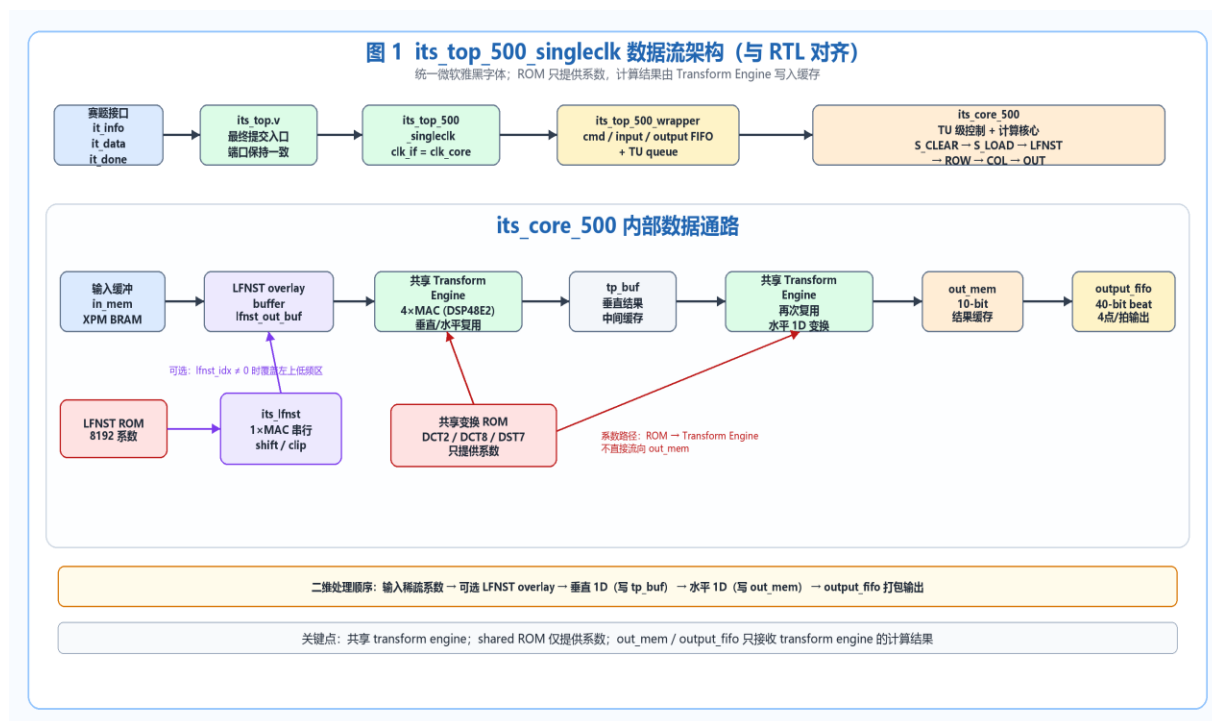
## 2. 系统架构设计

### 2.1 顶层架构框图

ITS 500MHz 单时钟实现顶层 its\_top\_500\_singleclk 的系统数据流架构如下所示。

顶层数据流分为配置接收、输入加载、可选 LFNST、垂直 1D 变换、水平 1D 变换和结果输出六个阶段。S\_ROW 在最终实现中承担垂直变换，按列读取输入缓冲；S\_COL 承担水平变换，按行读取中间缓冲。S\_ROW 阶段执行垂直变换（逐列读取 in\_mem），S\_COL 阶段执行水平变换（逐行读取 tp\_buf），处理顺序为先垂直后水平。

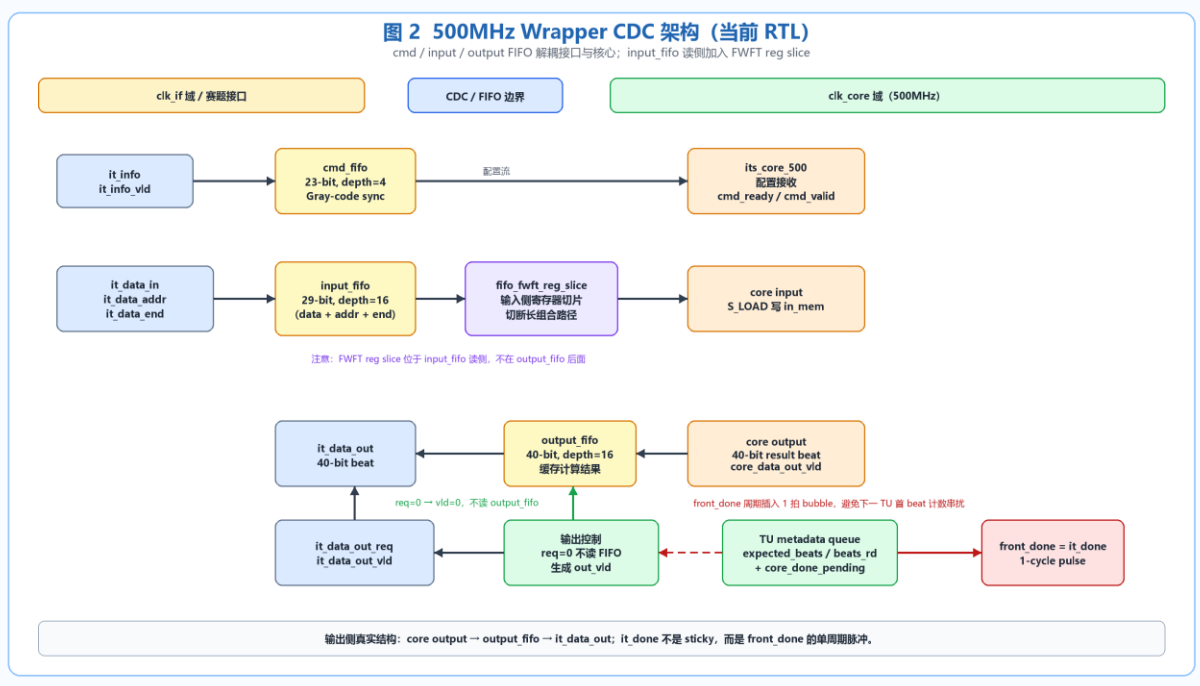
高频版本采用共享 transform engine，而不是为水平和垂直方向各保留一套计算阵列。垂直阶段和水平阶段严格串行，因此共享 4 路 MAC、共享 ROM 预取和共享系数缓存可以减少 DSP48E2 数量，同时不影响吞吐接口和功能覆盖。



## 2.2 500MHz Wrapper CDC 架构

its\_top\_500\_wrapper 采用双时钟域架构，在赛题单时钟接口与高速计算核心之间插入异步 FIFO CDC 层。

下图展示了跨时钟域的数据流和关键组件：



wrapper 版本用于说明完整系统集成时的 CDC 边界: cmd\_fifo 传递 TU 配置, input\_fifo 传递输入系数和 end marker, output\_fifo 缓冲计算结果。三个 FIFO 均采用 Gray-code 指针同步; 当前 RTL 中显式的 fifo\_fwft\_reg\_slice 位于 input\_fifo 读侧, 用于切断输入侧组合路径, output\_fifo 后不再额外插入该切片。

单时钟实现顶层 its\_top\_500\_singleclk 复用同一 wrapper/core 结构, 并将 clk\_if 与 clk\_core 同接到外部 clk。这样既保留了 CDC 设计的可扩展性, 又满足赛题单时钟接口要求, 并可用 2ns 约束直接评估 500MHz。赛题提交入口 02\_rtl/rtl/its\_top.v 内部例化 its\_top\_500\_singleclk, 端口与赛题 its\_top 接口保持一致。

连续 TU 输入能力的关键处理在 wrapper 侧完成。TU metadata queue 深度为 4, 用于保存已经进入计算核心但尚未完成输出的 TU 元信息; input closing 窗口在 end marker 写

入后短暂拉低 it\_data\_in\_req, 直到 input FIFO 安全越过当前 TU 边界, 避免下一 TU 的首个数据被当前 S\_LOAD 误读。

## 3. 状态机设计

### 3.1 顶层状态机

顶层 its\_top\_500\_singleclk / its\_core\_500 采用两级状态机架构, 主状态机控制 TU 级流程, 变换引擎内部状态机控制 1D 变换微流程。

主状态机只处理 TU 级大流程, 不直接展开每个乘累加周期。S\_LOAD 根据 it\_data\_end 或 FIFO end marker 判断输入结束; S\_LFNST 仅在 lfnst\_idx 非零时进入; S\_ROW\_START/S\_COL\_START 负责设置 transform engine 的方向、尺寸和变换类型; S\_OUT 负责按光栅顺序输出并响应反压。

在本项目中, 状态跳转同时承担协议保护职责: core 在 S\_LOAD 检测到 end 后立即停止继续读 input FIFO, wrapper 则通过 can\_accept\_tu 和 tuq\_full 共同控制新 TU 配置是否可进入, 避免数据流和配置流在连续 TU 场景下错位。

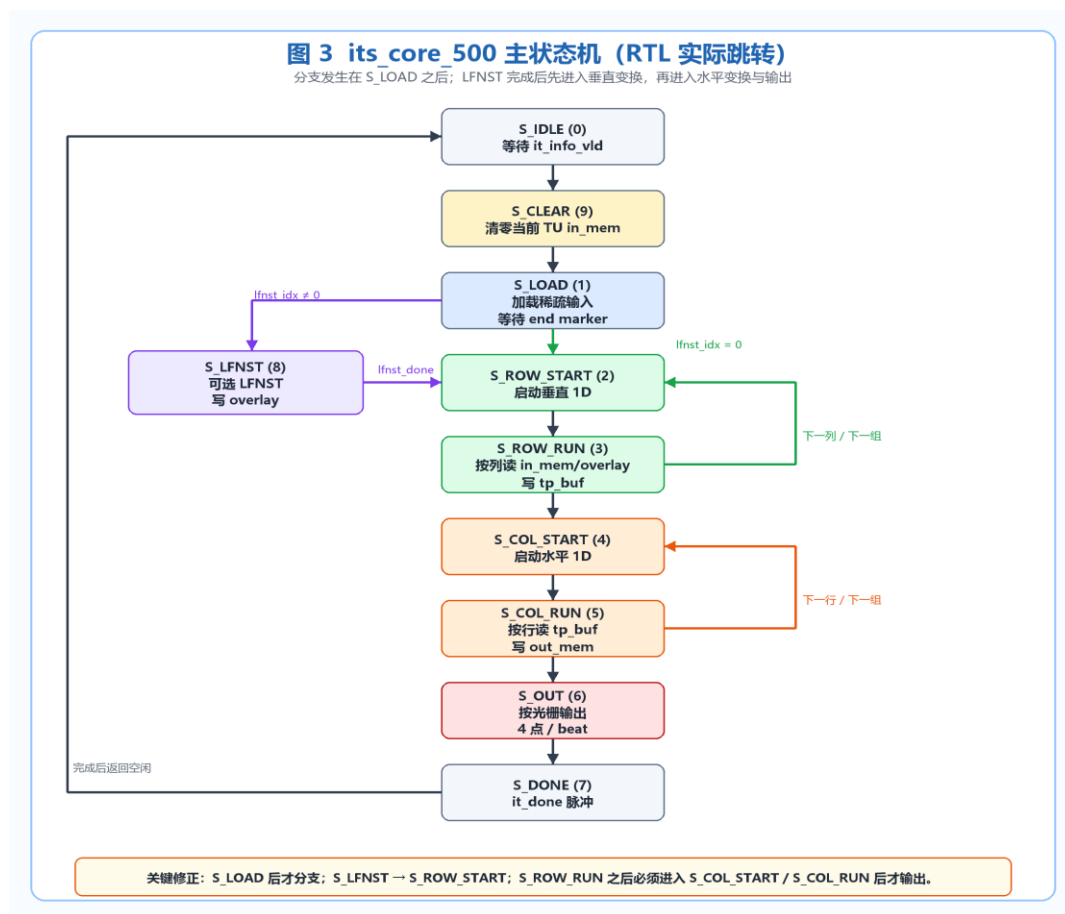
#### 主状态机状态定义:

状态定义: S\_ROW 表示垂直方向处理, 读取输入矩阵的列; S\_COL 表示水平方向处理, 读取 tp\_buf 的行。这样可以保持 RTL 状态编码稳定, 同时满足 VVC 规范中先垂直后水平的处理顺序。

| 状态 (编码)            | 功能描述                                     |
|--------------------|------------------------------------------|
| S_IDLE (4'd0)      | 空闲状态, 等待 it_info_vld 脉冲                  |
| S_CLEAR (4'd9)     | 清零 in_mem[0..total_points-1], 为下一 TU 做准备 |
| S_LOAD (4'd1)      | 加载输入系数, 外部按地址写入 in_mem, 只传非零点            |
| S_LFNST (4'd8)     | 执行 LFNST 反变换 (可选, lfnst_idx!=0 时进入)      |
| S_ROW_START (4'd2) | 垂直变换启动, 初始化变换引擎                          |

|                           |                                                            |
|---------------------------|------------------------------------------------------------|
| <b>S_ROW_RUN (4'd3)</b>   | 垂直变换执行, 逐列读取 in_mem/lfnst_out_buf overlay, 1D 变换后写入 tp_buf |
| <b>S_COL_START (4'd4)</b> | 水平变换启动                                                     |
| <b>S_COL_RUN (4'd5)</b>   | 水平变换执行, 按行读取 tp_buf, 1D 变换后写入 out_mem                      |
| <b>S_OUT (4'd6)</b>       | 生成输出 beat; 单个有效 beat 打包 4 个 10-bit 结果, 受 req/FIFO 反压控制。    |
| <b>S_DONE (4'd7)</b>      | core 计算完成; 最终 it_done 由 wrapper 在输出 beat 全部接收后产生。          |

### 状态转移流程图:



## 3.2 变换引擎内部状态机

its\_transform\_engine.v 内部 5 状态微状态机：

transform engine 内部按一组 4 个输出点组织计算。S\_PREFETCH 从变换 ROM 取 4 行系数写入 coeff\_buf, S\_COMPUTE 同时驱动 4 个 MAC 计算 4 个输出点, S\_OUTPUT 将累加结果执行  $(sum + 32) >>> 6$  后写回上层缓冲。

为满足 500MHz, ROM 地址生成使用累加器替代宽 barrel shifter, BRAM 到 DSP 的路径增加 P0 寄存器并保留必要的 DONT\_TOUCH, mac\_data\_r 做多份复制降低扇出。后续共享引擎版本进一步复用同一套 MAC, 使 DSP48E2 用量降到 5 个。

| 状态         | 功能                                                      |
|------------|---------------------------------------------------------|
| S_IDLE     | 等待 start 脉冲                                             |
| S_LOAD     | 加载 N 个输入到 line_buf (N 周期)                               |
| S_PREFETCH | 从 ROM 预取 4 行系数到 coeff_buf (4N 周期)                       |
| S_COMPUTE  | 4 MAC 并行计算 4 个输出点 (N 周期)                                |
| S_OUTPUT   | 生成输出 beat; 单个有效 beat 打包 4 个 10-bit 结果, 受 req/FIFO 反压控制。 |

循环机制：对于  $N > 4$  的变换, 将 N 行分成  $N/4$  组, 每组 4 行共享一次 ROM 预取。状态机循环 S\_PREFETCH  $\rightarrow$  S\_COMPUTE, 直到所有组处理完毕, 然后进入 S\_OUTPUT。

$N=4$  时只需一组 4 行系数,  $N=8/16/32/64$  时按  $N/4$  组循环。每组计算结束后更新 ROM 行索引、输入列索引和输出写地址, 直到当前方向的所有点处理完成。同步 ROM 和 BRAM 的读延迟通过预取状态和流水寄存器对齐。

## 3.3 LFNST 模块状态机

its\_lfnst.v 内部包含 8 个局部状态 (S\_IDLE、S\_LOAD、S\_PREFETCH、S\_COMPUTE、S\_DRAIN、S\_OUTPUT、S\_OUTPUT\_CLIP、S\_DONE) :

LFNST 只读取左上 4x4 的 16 个低频系数，输出数量由 nTrs 决定：任一边小于 8 时输出 16 个点，宽高均不小于 8 时输出 48 个点。nTrs=48 对应 3 个 4x4 子块，写入 lfnst\_out\_buf overlay，由后续垂直变换按地址优先读取。

LFNST MAC 链路包含输入缓存读、系数缓存读和 its\_mac 两级流水。为保证最后一个 x[15] 乘积完成累计，S\_DRAIN 等待 MAC 流水线排空后再捕获结果；随后 S\_OUTPUT 和 S\_OUTPUT\_CLIP 将加 64、右移 7、clip3 拆成两级，消除长 CARRY4 链。

| 状态                   | 功能                                                    |
|----------------------|-------------------------------------------------------|
| <b>S_IDLE</b>        | 等待 start                                              |
| <b>S_LOAD</b>        | 加载 16 个输入到 in_buf                                     |
| <b>S_PREFETCH</b>    | 从 ROM 预取 nTrs*16 个系数                                  |
| <b>S_COMPUTE</b>     | 16 周期内积计算（MAC 串行）                                     |
| <b>S_DRAIN</b>       | 等待 MAC 流水线排空（2 周期）                                    |
| <b>S_OUTPUT</b>      | 生成输出 beat；单个有效 beat 打包 4 个 10-bit 结果，受 req/FIFO 反压控制。 |
| <b>S_OUTPUT_CLIP</b> | 生成输出 beat；单个有效 beat 打包 4 个 10-bit 结果，受 req/FIFO 反压控制。 |
| <b>S_DONE</b>        | core 计算完成；最终 it_done 由 wrapper 在输出 beat 全部接收后产生。      |

## 4. 模块详细设计

### 4.1 顶层模块 (its\_top.v / its\_core\_500.v)

顶层模块负责 TU 级控制、内存管理和数据流调度。核心信号包括：

its\_core\_500 是高速计算核，使用 FIFO 风格接口与 wrapper 解耦；

its\_top\_500\_singleclk 是单时钟 500MHz 实现顶层，负责把赛题接口映射到 wrapper，并将接口时钟和核心时钟合并为单个 clk。

顶层模块需要同时维护输入地址、TU 尺寸、变换类型、LFNST 参数、输出计数和完成标志。输出完成不只由核心计算完成决定，还必须等待当前 TU 的所有有效输出 beat 被外部读取；wrapper 通过 TU metadata queue、beat 计数和 front\_done 条件产生最终 it\_done，确保 it\_done 不会早于最后一拍有效数据。

| 信号名                | 位宽 | 方向 | 说明                                                                       |
|--------------------|----|----|--------------------------------------------------------------------------|
| clk                | 1  | I  | 赛题单时钟接口时钟 (500MHz) 。<br>clk_core 仅用于双时钟 wrapper 集成参考，不属于最终 its_top.v 端口。 |
| rst_n              | 1  | I  | 异步复位，低有效                                                                 |
| it_info[21:0]      | 22 | I  | TU 配置信息总线                                                                |
| it_info_vld        | 1  | I  | 信息有效脉冲                                                                   |
| it_data_in[15:0]   | 16 | I  | 输入系数（有符号，只送非零点）                                                          |
| it_data_addr[11:0] | 12 | I  | 系数在 TU 内的光栅扫描地址                                                          |
| it_data_in_vld     | 1  | I  | 输入数据有效                                                                   |
| it_data_end        | 1  | I  | 输入数据结束脉冲                                                                 |
| it_data_in_req     | 1  | O  | 输入请求反压                                                                   |
| it_data_out[39:0]  | 40 | O  | 输出结果（4×10-bit 有符号拼接）                                                     |
| it_data_out_vld    | 1  | O  | 输出有效                                                                     |
| it_data_out_req    | 1  | I  | 输出反压                                                                     |
| it_done            | 1  | O  | TU 处理完成脉冲                                                                |

### it\_info[21:0] 位域定义:

tr\_type\_hor 和 tr\_type\_ver 分别控制水平与垂直方向的 1D 变换核。垂直方向先执行, S\_ROW 使用 tr\_type\_ver 和 tu\_height 完成垂直变换, S\_COL 使用 tr\_type\_hor 和 tu\_width 完成水平变换。lfnst\_tr\_set\_idx 与 lfnst\_idx 只在 S\_LFNST 阶段生效。当 lfnst\_idx != 0 时, S\_LFNST 后的主二维反变换强制使用 DCT2, 即 row\_tr\_type=0、col\_tr\_type=0; 当 lfnst\_idx == 0 时, S\_ROW 使用 tr\_type\_ver, S\_COL 使用 tr\_type\_hor。

| 位域      | 信号名              | 说明                              |
|---------|------------------|---------------------------------|
| [6:0]   | tu_width         | TU 宽度 (4/8/16/32/64)            |
| [13:7]  | tu_height        | TU 高度 (4/8/16/32/64)            |
| [15:14] | tr_type_hor      | 水平变换类型 (0=DCT2, 1=DCT8, 2=DST7) |
| [17:16] | tr_type_ver      | 垂直变换类型                          |
| [19:18] | lfnst_tr_set_idx | LFNST 变换集索引 (0..3)              |
| [21:20] | lfnst_idx        | LFNST 核索引 (0=不启用, 1/2)          |

## 4.2 变换引擎 (its\_transform\_engine.v)

变换引擎是 ITS 的核心计算单元, 执行一维反变换  $y = T^T * x$ 。采用 4 MAC 并行架构, 每周期可计算 4 个输出点。

系数 ROM 以 size、transform type、row 和 col 组成地址。改为 ROM 地址累加器, 缩短组合逻辑。UltraScale+ 上最差路径最终转移到 ROM 输出写入 coeff\_buf 的 0 级逻辑物理路径。

line\_buf、coeff\_buf 和 result\_buf 均按高频访问模式布置。line\_buf 保存当前 1D 输入向量, coeff\_buf 保存 4 行系数窗口, result\_buf 暂存 4 路 MAC 输出。对于大尺寸 TU, engine 以组为单位反复预取和计算, 避免一次展开过多 MAC 造成面积上升。

### 计算流程 (N 点变换) :

- ① S\_LOAD: 加载 N 个输入到 line\_buf (N 周期)
- ② S\_PREFETCH: 从 ROM 预取 4 行系数到 coeff\_buf (4N 周期)
- ③ S\_COMPUTE: 4 MAC 并行计算 4 个输出点 (N 周期)
- ④ 若  $N > 4$ , 重复步骤②-③, 共  $N/4$  组
- ⑤ S\_OUTPUT: 输出结果, 量化公式  $(sum + 32) >>> 6$

## 4.3 MAC 单元 (its\_mac.v)

2 级流水线乘累加器结构:

its\_mac 将乘法和累加拆成两级: 第一拍产生  $16 \times 16$  有符号乘积, 第二拍与累加器相加。clr 用于每个输出点开始时清零累加器, en 用于控制有效乘积进入流水。高频版本把 mac\_clr 注册化, 避免控制信号从预取状态直接穿过长组合链。

| 流水级          | 运算                                                | 位宽                                       |
|--------------|---------------------------------------------------|------------------------------------------|
| Stage 1 (乘法) | $product = signed(a) * signed(b)$                 | $16 \times 16 \rightarrow 32\text{-bit}$ |
| Stage 2 (累加) | $result += signed(\{8\{product[31]\}, product\})$ | 40-bit accumulator                       |
| 控制信号         | en: 使能乘法   clr: 清零累加器 (优先级高于累加)                   |                                          |

## 4.4 LFNST 模块 (its\_lfnst.v)

LFNST 模块执行低频次不可分变换, 公式:  $y[i] = clip3(-32768, 32767, (\sum_j T[i][j] \cdot x[j] + 64) >> 7)$

LFNST 与主变换的不同点在于矩阵不可分，不能拆成水平和垂直两次 1D 变换。因此模块串行遍历 16 个输入，对每个输出行执行 16 项内积。虽然吞吐低于 transform engine，但 LFNST 只作用于低频区域，整体延迟占比可控。

overlay buffer 的作用是避免把 LFNST 结果重新写回大容量 in\_mem。后续垂直变换读取输入矩阵时，如果地址落在 LFNST 覆盖区域，则优先使用 lfnst\_out\_buf；否则使用原 in\_mem。这样减少了清零和写回路径，也降低了 in\_mem 端口压力。nTrs=16 场景下，LFNST 输入和 overlay 覆盖的左上 4×4 不是 flat in\_mem[0..15]，而是按二维坐标访问： $addr = row \times tu\_width + col$ 。该映射可正确覆盖 8×4、16×4、32×4、64×4 等非方形 TU。

#### nTrs 定义：

| 条件                                | nTrs | 矩阵大小  | 输入数 | 输出数 | 写回区域       |
|-----------------------------------|------|-------|-----|-----|------------|
| tu_width<8   <br>tu_height<8      | 16   | 16×16 | 16  | 16  | 左上 4×4     |
| tu_width>=8<br>&&<br>tu_height>=8 | 48   | 48×16 | 16  | 48  | 3 个 4×4 子块 |

#### nTrs=48 子块布局 (写入 lfnst\_out\_buf overlay, 共 3 个 4×4 子块)：

|                                                                                                                                    | cols 0-3       | cols 4-7       | cols 8+ |
|------------------------------------------------------------------------------------------------------------------------------------|----------------|----------------|---------|
| rows 0-3                                                                                                                           | blk 0<br>(4×4) | blk 1<br>(4×4) | 保留      |
| rows 4-7                                                                                                                           | blk 2<br>(4×4) | 保留             | 保留      |
| rows 8+                                                                                                                            | 保留             | 保留             | 保留      |
| 地址映射：blk0: $addr = row \times width + col$   blk1: $addr = row \times width + (col+4)$   blk2: $addr = (row+4) \times width + col$ |                |                |         |

LFNST 输出流水线: S\_DRAIN (2 周期等待 MAC 排空) → S\_OUTPUT

((captured\_result+64)>>>7 写入 shifted\_r) → S\_OUTPUT\_CLIP (clip3 裁剪, 输出数据)。该流水线拆分消除了 12 级 CARRY4 关键路径。

nTrs=16 和 nTrs=48 共用同一条计算流水, 仅输出地址映射不同。nTrs=16 覆盖左上一个 4x4 子块, nTrs=48 覆盖三个 4x4 子块。测试集中专门包含非零 x[15] 的 LFNST 用例, 用于验证 MAC drain 对最后一个输入项的处理。

## 4.5 异步 FIFO CDC 模块

异步 FIFO (async\_fifo.v) 关键设计:

FIFO 采用 registered full flag, 写入条件统一为 wr\_en 与非 full 的握手结果, 避免组合 full 直接参与写使能造成毛刺。读侧为 First Word Fall Through, empty 为 0 时数据保持稳定, 只有 rd\_en 成功时才推进读指针。

在 wrapper 与 core 之间增加 fifo\_fwft\_reg\_slice 后, FIFO 读指针到 core in\_mem 写端的组合路径被切断。后续又将 in\_mem 在综合下显式映射为 xpm\_memory\_sdpram, 消除了大规模 DistRAM mux 树, wrapper OOC 时序由负裕量转为正裕量。

| 设计点  | 实现方案                                           |
|------|------------------------------------------------|
| 写指针  | 二进制计数器, 转换为 Gray-code 后同步到读时钟域                 |
| 读指针  | 二进制计数器, 转换为 Gray-code 后同步到写时钟域                 |
| 满标志  | Registered full flag, 避免组合逻辑满标志的毛刺             |
| 空标志  | 组合逻辑比较 Gray-code 指针                            |
| FWFT | First Word Fall Through, 数据在 empty=0 时直接出现在输出  |
| 深度   | cmd_fifo: 4   input_fifo: 16   output_fifo: 16 |

## 5. 数据流与时序设计

### 5.1 完整数据流

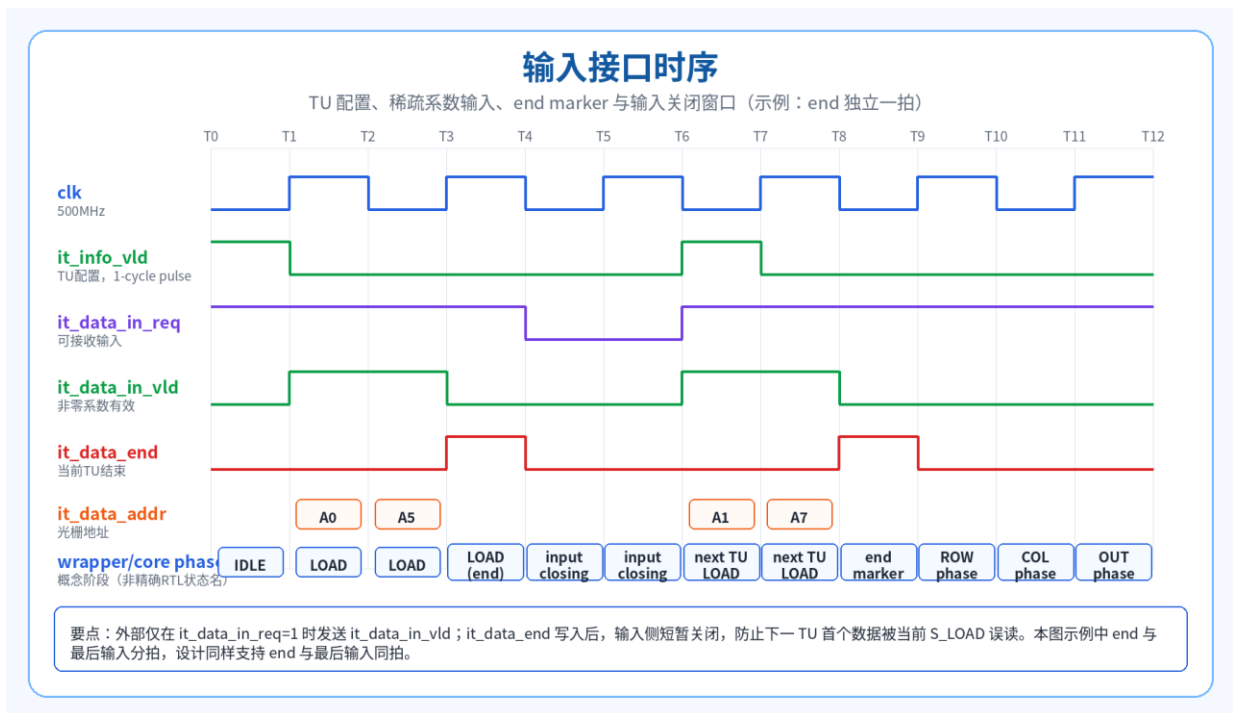
- ① 输入阶段：外部通过 `it_data_in` / `it_data_addr` / `it_data_in_vld` 写入输入缓冲 `in_mem`。
- ② LFNST 阶段：若 `lfnst_idx!=0`，读取 `in_mem` 左上  $4 \times 4$ ，经 LFNST 后写入 `lfnst_out_buf` overlay。
- ③ 垂直变换阶段：逐列读取 `in_mem/overlay`；若 `lfnst_idx=0`，则按 `tr_type_ver` 和 `tu_height` 做 1D 变换，若 `lfnst_idx!=0`，则强制使用 DCT2；结果写入 `tp_buf`。
- ④ 水平变换阶段：按行读取 `tp_buf`；若 `lfnst_idx=0`，则按 `tr_type_hor` 和 `tu_width` 做 1D 变换，若 `lfnst_idx!=0`，则强制使用 DCT2；结果写入 `out_mem`。
- ⑤ 输出阶段：按光栅扫描顺序从 `out_mem` 读取，每周期输出 4 个 10-bit 结果 (`it_data_out[39:0]`)。

输入数据采用稀疏传输方式，外部只发送非零系数及其 TU 内地址。S\_CLEAR 在每个 TU 前清理当前尺寸范围内的输入缓冲，保证未发送的位置默认为 0。对于连续 TU 场景，清零、输入关闭窗口和 metadata queue 共同保证前后 TU 的数据不会互相污染。

二维变换的数据方向在数据流中非常关键。垂直阶段按列读取输入矩阵，输出写入 `tp_buf`；水平阶段再按行读取 `tp_buf`，最终写入 `out_mem`。这个顺序与 reference model 保持一致，满足赛题要求的先垂直后水平处理顺序。

### 5.2 接口时序图

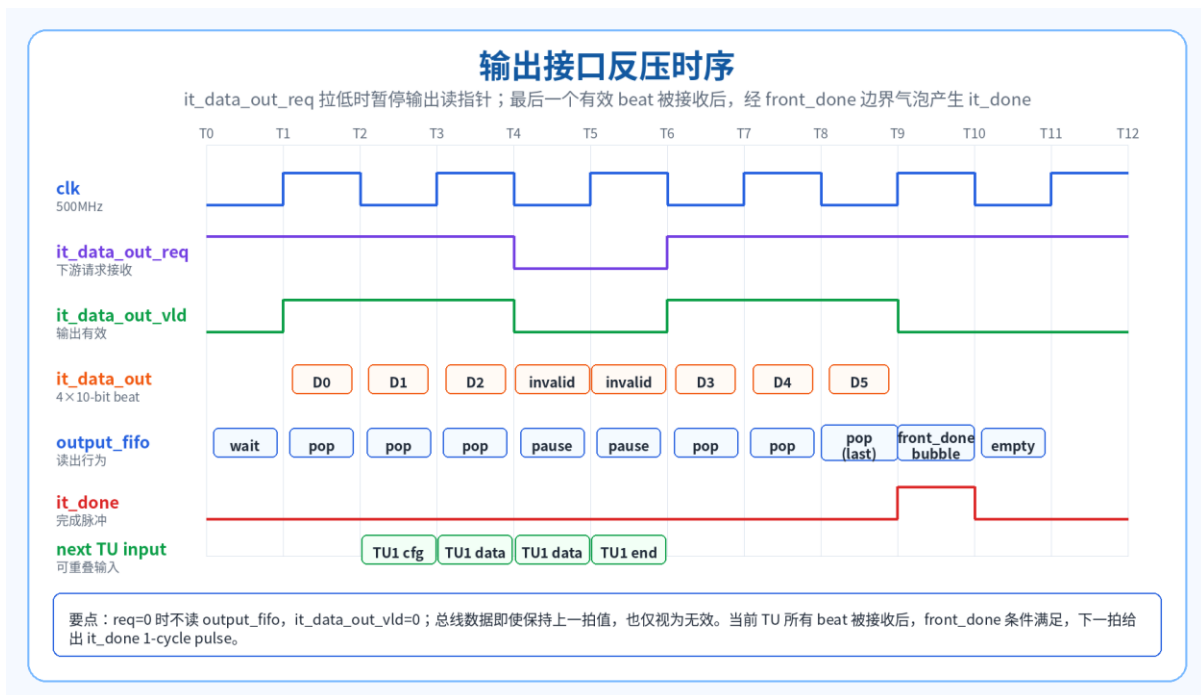
输入接口时序如下图所示。图中示例采用 `it_data_end` 独立一拍的形式；当前 RTL 同样支持 `it_data_end` 与最后一个 `it_data_in_vld` 同拍。



说明：T0 周期 `it_info_vld` 传入当前 TU 配置；外部仅在 `it_data_in_req=1` 时发送 `it_data_in_vld`。`it_data_end` 写入后，输入侧短暂关闭形成 input closing 窗口，防止下一 TU 首个数据被当前 S\_LOAD 误读。

`it_data_in_req` 为输入侧反压信号，表示当前顶层是否可以接收输入数据。一般情况下，在 S\_LOAD 期间保持为 1；当 FIFO 接近满、TU metadata queue 满，或刚写入 end marker 需要关闭当前 TU 输入窗口时会拉低。外部应只在 req 为 1 时发送有效输入。

输出接口反压时序如下图所示。



说明：it\_data\_out\_req=0 时暂停读取 output\_fifo，it\_data\_out\_vld 拉低；总线数据即使保持上一拍值，也仅视为无效。req 恢复后继续弹出后续 beat。

输出侧采用 req 门控读取 output\_fifo。当前 TU 的最后一个有效 beat 被接收后，wrapper 在 front\_done 边界插入 1 拍 bubble，随后产生 it\_done 单周期脉冲，避免下一 TU 的首 beat 与当前 TU 的 beat 计数串扰。

回归测试覆盖无反压、周期性反压、长反压、连续 TU 和 immediate overlap TU。immediate overlap 用例会在 TU0 输入结束后，只要 it\_data\_in\_req 允许就立即发送 TU1，用于验证赛题要求中的输入输出并行能力。

## 6. ROM 结构设计

### 6.1 变换核 ROM (its\_rom.v)

8176 条目，16-bit 有符号系数，同步读（1 周期延迟）。

变换核 ROM 覆盖 DCT2、DCT8 和 DST7 的所有支持尺寸。DCT2 支持 4x4 到 64x64，DCT8/DST7 支持 4x4 到 32x32。base\_addr 根据变换类型和尺寸选择，row/col 由 transform engine 状态机产生。

ROM 为同步读结构，地址拍和数据拍之间存在固定延迟。高频实现中通过预取状态隐藏该延迟，并把地址计算从组合乘法改为移位、累加和寄存器锁存，减少扇出和路由压力。

| 变换类型 | 块大小 | 起始地址 | 系数数量 |
|------|-----|------|------|
| DCT2 | 4   | 0    | 16   |
| DCT2 | 8   | 16   | 64   |
| DCT2 | 16  | 80   | 256  |
| DCT2 | 32  | 336  | 1024 |
| DCT2 | 64  | 1360 | 4096 |
| DCT8 | 4   | 5456 | 16   |
| DCT8 | 8   | 5472 | 64   |
| DCT8 | 16  | 5536 | 256  |
| DCT8 | 32  | 5792 | 1024 |
| DST7 | 4   | 6816 | 16   |
| DST7 | 8   | 6832 | 64   |
| DST7 | 16  | 6896 | 256  |
| DST7 | 32  | 7152 | 1024 |

地址公式：addr = base\_addr + row \* size + col

系数生成：DCT2 使用  $89 \cdot \cos$  量化，DCT8/DST7 使用  $64 \cdot \cos/\sin$  量化，均取最接近整数。

## 6.2 LFNST ROM (its\_lfnst\_rom.v)

8192 条目，16-bit 有符号系数，同步读（1 周期延迟）。

LFNST ROM 地址由 setIdx、idx、nTrs 模式、输出行和输入列组成。nTrs=16 区域占 2048 项，nTrs=48 区域占 6144 项，总计 8192 项。ROM 文件与测试模型共用同一生成流程，防止仿真 ROM 和参考模型系数不一致。

| 地址范围         | 模式             | 说明                                                                                |
|--------------|----------------|-----------------------------------------------------------------------------------|
| [0..2047]    | <b>nTrs=16</b> | $4 \text{ setIdx} \times 2 \text{ idx} \times 16 \times 16 = 8 \times 256 = 2048$ |
| [2048..8191] | <b>nTrs=48</b> | $4 \text{ setIdx} \times 2 \text{ idx} \times 48 \times 16 = 8 \times 768 = 6144$ |

## 7. 定点量化方案

### 7.1 量化公式与位宽

| 模块           | 运算                          | 量化方式                                                        | 位宽         |
|--------------|-----------------------------|-------------------------------------------------------------|------------|
| <b>变换引擎</b>  | $y = T * x$                 | $(\text{sum} + 32) >>> 6$                                   | 40-bit 累加器 |
| <b>LFNST</b> | $y = T * x$                 | $(\text{sum} + 64) >>> 7,$<br>$\text{clip3}(-32768, 32767)$ | 40-bit 累加器 |
| <b>最终输出</b>  | 40-bit $\rightarrow$ 10-bit | 截取低 10 位                                                    | 10-bit 有符号 |

本设计采用全整数定点运算实现反变换，各级数据通路位宽与量化定标定义如下。

输入系数 `it_data_in` 为 16-bit 有符号整数。变换核 ROM 系数同样以 16-bit 有符号格式存储：DCT2 核采用  $89 \cdot \cos(\cdot)$  定标，DCT8/DST7 核采用  $64 \cdot \cos/\sin(\cdot)$  定标，LFNST 核采用  $128 \cdot T[i][j]$  定标，均取最接近的整数值。该系数生成方式与赛题附件及 Python reference model 保持一致，避免 RTL 与 golden 数据之间出现系统性偏差。

中间乘累加运算采用 40-bit 有符号累加器，以容纳最坏情况下 64 项乘积之和（溢出分析详见 7.2 节）。

输出级量化公式如下：

1D 变换引擎：单点乘累加完成后执行  $(\text{sum} + 32) >>> 6$ ，实现除以 64 的四舍五入，结果写入 `tp_buf` (16-bit)。

LFNST 模块：执行  $(\text{sum} + 64) >>> 7$ ，实现除以 128 的四舍五入，再经  $\text{clip3}(-32768, 32767)$  裁剪为 16-bit，写入 `lfnst_out_buf`。

最终输出：2D 反变换完成后从 40-bit 累加结果截取低 10-bit 有符号数，按光栅扫描顺序每拍打包 4 点输出。

上述各级量化点与 reference model 完全对齐，回归测试采用逐 beat 比对验证。

## 7.2 溢出分析

最坏情况分析（所有输入取最大值，系数取最大值）：

| 变换类型  | 最大 T | 最大 x  | 最大乘积      | N  | 最大累加和       | 所需位宽   |
|-------|------|-------|-----------|----|-------------|--------|
| DCT2  | 89   | 32768 | 2,916,352 | 64 | 186,646,528 | 28 bit |
| DCT8  | 64   | 32768 | 2,097,152 | 32 | 67,108,864  | 27 bit |
| DST7  | 64   | 32768 | 2,097,152 | 32 | 67,108,864  | 27 bit |
| LFNST | 128  | 32768 | 4,194,304 | 16 | 67,108,864  | 27 bit |

40-bit 累加器有效范围  $2^{39} = 549,755,813,887$ ，远超最大累加和 186,646,528，无溢出风险。

40-bit 不是由单次乘积决定，而是为最坏情况下 64 项累加和两级变换留出余量。即使输入取 16-bit 极值、系数取最大量化值，DCT2 的 64 项累加仍远低于 40-bit 有符号范围，因此 MAC 内部不需要饱和处理。

## 7.3 量化误差

变换引擎  $>>> 6$  引入的量化误差  $\leq 0.5$  LSB（+32 实现四舍五入）。2D 反变换两次量化误差理论上较小；本文不单独进行图像级 PSNR 评估，功能正确性以 RTL 与 Python golden 的逐 beat bit-exact 回归结果作为依据。

最终输出截取为 10-bit 有符号结果，与赛题接口 `it_data_out[39:0]` 的 4 路拼接格式一致。LFNST 输出在写入 overlay 前先 clip 到 16-bit，主 2D 反变换输出再按 10-bit 格式返回。各级量化点、截取/打包规则与 reference model 一致，回归测试直接逐 beat 比对 golden。

## 8. 存储资源汇总

存储结构按访问频率分层设计：in\_mem、tp\_buf、out\_mem 保存 TU 级矩阵数据，line\_buf、coeff\_buf、result\_buf 服务单次 1D 变换，LFNST 的 in\_buf、coeff\_buf、lfnst\_out\_buf 只服务低频区域。高频路径优先使用 BRAM/XPM 和寄存器切片，低容量局部缓存则保留为分布式 RAM。

综合版本中 in\_mem 使用 XPM\_MEMORY\_SDP RAM 推断为 2 个 RAMB36E2；仿真版本使用 reg array，避免 ModelSim 10.6e 对 XPM 库支持不足。该条件编译只改变存储实现形式，不改变读写协议和功能结果。

| 存储名               | 深度   | 位宽     | 容量          | 用途                                 |
|-------------------|------|--------|-------------|------------------------------------|
| in_mem            | 4096 | 16-bit | 65,536 bits | 输入系数缓冲（综合 XPM BRAM / 仿真 reg array） |
| tp_buf            | 4096 | 16-bit | 65,536 bits | 转置缓冲（垂直变换输出）                       |
| out_mem           | 4096 | 10-bit | 40,960 bits | 输出结果缓冲                             |
| line_buf          | 64   | 16-bit | 1,024 bits  | 变换引擎行输入缓冲                          |
| coeff_buf         | 256  | 16-bit | 4,096 bits  | 变换引擎系数缓冲                           |
| result_buf        | 64   | 40-bit | 2,560 bits  | 变换引擎结果缓冲                           |
| in_buf (LFNST)    | 16   | 16-bit | 256 bits    | LFNST 输入缓冲                         |
| coeff_buf (LFNST) | 768  | 16-bit | 12,288 bits | LFNST 系数缓冲                         |
| lfnst_out_buf     | 48   | 16-bit | 768 bits    | LFNST 结果覆盖缓冲 (its_core_500)        |

## 9. 目标频率分析与实现结果

### 9.1 UltraScale+ 500MHz 实现结果

单时钟实现顶层 its\_top\_500\_singleclk, 在 Kintex UltraScale+ xcku5p-ffvb676-2-e 上的 OOC 综合结果如下; 功能回归 its\_top\_500\_singleclk 1539/1539 PASS, its\_core\_500 94/94 PASS, 已覆盖赛题要求中的先垂直后水平处理, 以及上一 TU 输出未完成时接收下一 TU 的连续输入能力。

综合约束为 2.000ns 时钟周期。实现报告显示 WNS +0.087ns、WHS +0.035ns、TNS 0、Failing Endpoints 0, 说明 setup 和 hold 均已闭合。资源使用为 CLB LUT 1886、CLB Register 2214、DSP48E2 5、RAMB36E2/RAMB18E2 12/5, 资源占比很低。

最差路径为 shared ROM 到 shared transform engine coeff\_buf 的物理写入路径, 逻辑级数为 0。该结果说明设计中的较长组合逻辑路径已通过流水线和存储结构优化得到控制, 当前时序裕量主要由 BRAM 输出、局部路由和 RAMD64E 写入端决定。由于 slack 为正, 该路径满足 500MHz 时序约束。

| 指标                         | 值             | 状态  |
|----------------------------|---------------|-----|
| <b>WNS (Setup)</b>         | +0.087 ns     | MET |
| <b>TNS</b>                 | 0.000 ns      | —   |
| <b>WHS (Hold)</b>          | +0.035 ns     | MET |
| <b>Failing Endpoints</b>   | 0             | —   |
| <b>CLB LUTs</b>            | 1,886 (0.87%) | —   |
| <b>CLB Registers</b>       | 2,214 (0.51%) | —   |
| <b>DSP48E2</b>             | 5             | —   |
| <b>RAMB36E2 / RAMB18E2</b> | 12 / 5        | —   |

Worst Path: shared ROM coeff\_reg\_1 (RAMB36E2) → shared transform engine coeff\_buf (RAMD64E/DistRAM), data path delay 1.789ns, 0 级逻辑。该路径为 ROM 输出写入系数缓存的物理路径；WNS 仍为正, 500MHz 已达标。

| 项目              | 数据                                                | 说明                                           |
|-----------------|---------------------------------------------------|----------------------------------------------|
| 目标器件            | Kintex UltraScale+ xcku5p-ffvb676-2-e             | FPGA PPA 评估唯一器件型号                            |
| 最差 setup 路径     | shared ROM coeff_reg_1 -> shared engine coeff_buf | 0 级逻辑, WNS +0.087ns                          |
| Data Path Delay | 1.789 ns                                          | logic 0.960ns / route 0.829ns                |
| 最差 hold 路径      | output_fifo_wdata_reg[38] -> output_fifo RAMD32   | WHS +0.035ns                                 |
| 结论              | 500MHz MET                                        | setup/hold/pulse width 均 0 failing endpoints |

## 9.2 FPGA 评估口径说明

本项目 PPA 评估采用 FPGA 工具实现报告, 不使用 ASIC 工艺库进行估计。报告中唯一目标器件为 Kintex UltraScale+ xcku5p-ffvb676-2-e, 相关时序、面积和功耗均以 Vivado 2025.2 post-route/OOC 报告为依据。

## 10. RTL 源代码清单

代码组织上, 02\_rtl/rtl/ 目录包含最终提交所需的所有硬件模块, 03\_verification/tb/ 目录包含回归测试向量和 testbench, 03\_verification/scripts/ 目录保存 reference model、golden 生成和报告辅助脚本, 02\_rtl/synth/ 目录保存 Vivado OOC 综合脚本和实现报告。

最终交付以 02\_rtl/rtl/its\_top.v 为赛题提交入口, 同时提交 its\_top\_500\_singleclk.v、ROM hex、LFNST hex、仿真脚本、综合脚本和验证报告。its\_top\_500\_wrapper.v 作为系统集成参考, 说明当接口时钟与核心时钟分离时如何通过异步 FIFO 完成 CDC。

| 文件名                            | 说明                                                |
|--------------------------------|---------------------------------------------------|
| <b>its_top.v</b>               | 赛题提交顶层封装入口（内部例化 its_top_500_singleclk, 端口与赛题接口一致） |
| <b>its_top_500_singleclk.v</b> | 单时钟 500MHz 实现顶层（由 its_top.v 作为赛题提交入口进行封装）         |
| <b>its_top_500_wrapper.v</b>   | 双时钟 CDC wrapper（保留用于系统集成验证）                       |
| <b>its_core_500.v</b>          | 500MHz 计算核（FIFO 接口）                               |
| <b>its_pkg.v</b>               | 共享 package（状态编码 + 位移乘法函数）                         |
| <b>its_transform_engine.v</b>  | 1D 变换引擎（4 MAC 并行）                                 |
| <b>its_mac.v</b>               | 流水线乘累加单元                                          |
| <b>its_lfnst.v</b>             | LFNST 反变换模块                                       |
| <b>its_rom.v</b>               | 变换核 ROM（8176 系数）                                  |
| <b>its_lfnst_rom.v</b>         | LFNST 系数 ROM（8192 系数）                             |
| <b>async_fifo.v</b>            | Gray-code 异步 FIFO（CDC）                            |
| <b>rst_sync.v</b>              | 复位同步器                                             |
| <b>fifo_fwft_reg_slice.v</b>   | FWFT 寄存器切片                                        |